TP: Implementation of Data Assimilation Networks

Compte-rendu

Paul Louka

April 2026

The project code is available at `https://github.com/latestdesign/data_assim_networks`. *The theoretical background presented in this report follows the course material by Sixin Zhang (*`sixin.zhang@toulouse-inp.fr`*).*

1 Introduction

Data Assimilation Networks (DAN) combine the principles of Bayesian data assimilation with recurrent neural network architectures to estimate the hidden state of an observed dynamical system. Given a state-space model

$$\begin{aligned} x_t &= M(x_{t-1}) + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q) \\ y_t &= Hx_t + \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, R) \end{aligned}$$

the DAN learns three modules: a propagator $\mathbf{b}$ that propagates the latent hidden state forward in time, an analyser $\mathbf{a}$ that fuses the predicted hidden state with the current observation y_t, and a decoder $\mathbf{c}$ that maps the hidden state to a Gaussian distribution over the true state x_t.
Thus, at each time step, the DAN computes

$$h^b_{t+1} = \mathbf{b}(h^a_t), \qquad h^a_{t+1} = \mathbf{a}(h^b_{t+1}, y_{t+1}), \qquad q^b_t = \mathbf{c}(h^b_t), \qquad q^a_t = \mathbf{c}(h^a_t)$$

producing prediction and analysis distributions q^b_t and q^a_t. The training loss is a function of the negative log-likelihoods of the true state x_t under each distribution.

This report presents two experiments: full training on a Linear 2D system, and online training on the chaotic Lorenz 96 system in 40 dimensions. The key architectural difference between the two training modes is how gradients flow through time. Both modes rely on backpropagation through time (BPTT). Full training with L-BFGS performs full BPTT: it unrolls the entire trajectory and optimizes a single closure, allowing gradients to flow through all T steps. Online training with Adam performs truncated BPTT, so that gradients only flow through a single step.

2 Linear 2D: Full Training

2.1 System and Setup

The 2D linear system consists of rotation dynamics $M = R_\theta$ with $\theta = \pi/100$, which produces a periodic orbit. The state dimension is $n = 2$, with batch size mb $= 128$, hidden dimension

$h_{\text{dim}} = 4$, and noise scales $\sigma_0 = \sigma_p = 0.01$, $\sigma_o = 0.1$. This ties directly to $\eta_t \sim \mathcal{N}(0, \sigma_p^2 I)$ and $\epsilon_t \sim \mathcal{N}(0, \sigma_o^2 I)$ from the introduction. The initial state x_0 is sampled from $\mathcal{N}(3 \cdot \mathbf{1}, \sigma_0^2 I)$, so the system starts near the point $(3, 3)$ and rotates around the origin.

2.2 Pre-training of c

TODO 1.1. The function `experiment` in `manage_exp.py` manages the full pipeline: it reads the configuration dictionary, constructs the DAN, the propagator (`ConstructorProp`), and the observer (`ConstructorObs`), then calls the appropriate training functions. When full training is selected, it first pre-trains the decoder $\mathbf{c}$ using the initial state x_0 and the initial distribution q_0^a from the `Gaussian` module in `ConstructorC`. This step helps the decoder learn a good initial mapping from hidden states to Gaussian parameters before joint training of all modules.

TODO 1.2. The `Gaussian` module in `ConstructorC` converts a vector v into the parameters (μ, Λ) of a Gaussian distribution $\mathcal{N}(\mu, \Sigma = \Lambda\Lambda^\top)$. The diagonal entries of Λ are exponentiated from the vector, with clamping to $[-8, 8]$ before exponentiation to prevent numerical overflow. The log-probability is computed as

$$\log p(x) = -\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu) - \sum_i \left(\frac{\log 2\pi}{2} + \log \Lambda_{ii} \right),$$

where Σ^{-1} is obtained via `torch.cholesky_inverse`, since Σ is positive definite by construction.

TODO 1.3. The initial state x_0 is drawn from $\mathcal{N}(3 \cdot \mathbf{1}, \sigma_0^2 I)$ and stored as a matrix of shape (mb, n). The `Lin2d` module implements Mx where $M = R_\theta$ is the 2×2 rotation matrix, applied on the entire batch via `self.M @ x.T`.

TODO 1.4.

```
Pre-train c at t=0
empirical mean of x0 is tensor([2.9997, 3.0001])
## INIT a0 mean tensor([2.9997, 3.0001])
## INIT a0 var tensor([1.0304e-04, 7.9643e-05])
## INIT a0 covar tensor([[1.0304e-04, 1.7344e-06], [1.7344e-06, 7.9672e-05]])
```

The `closure0` function zeros the gradients, computes $L_0(q_0^a) = -\log p_{q_0^a}(x_0)$ averaged over the batch, and calls `backward()`. After optimization with L-BFGS, we observe that the empirical mean of x_0 is approximately $(3.0, 3.0)$, matching the true mean. The initial variance is around 10^{-4}, which is close to $\sigma_0^2 = 10^{-4}$, indicating that the decoder has successfully learned to represent the initial distribution. The covariance matrix is nearly diagonal, which makes sense given we are modeling a state with independent noise in each dimension.

TODO 1.5. The `Gaussian` module originally looped over each sample in the batch to compute the covariance $\Sigma = \Lambda\Lambda^\top$, its inverse, and the log of the determinant. This was replaced by:

- `torch.bmm(Lambda, Lambda.transpose(1,2))` for the batched covariance $\Sigma = \Lambda\Lambda^\top$,
- `torch.cholesky_inverse(Lambda)` which exploits the triangular structure of Λ to compute Σ^{-1} directly,
- a batched `torch.sum` over the diagonal entries for the second term, since $\log|\Sigma| = 2\sum_i \log \Lambda_{ii}$,
- `unsqueeze` reshaping of $(x-\mu)$ into row and column matrices for the quadratic form $(x-\mu)^\top \Sigma^{-1}(x-\mu)$, allowing a single batched matrix product.

2.3 Full Training of a, b, c

TODO 2.1. The error in `FcZero` was that `self.alphas` was a classic tensor instead of an `nn.Parameter`. Without this, the ReZero scaling coefficients α are not learnable, and the residual connections $h + \alpha \cdot \mathrm{act}(\mathrm{lin}(h))$ would be frozen.

TODO 2.2. The DAN forward pass implements the cycle: $h^b = \mathbf{b}(h^a)$, $q^b = \mathbf{c}(h^b)$, $h^a = \mathbf{a}([h^b, y])$, $q^a = \mathbf{c}(h^a)$.

TODO 2.3. Training sequences are generated by iterating $x_{t+1} \sim \mathrm{prop}(x_t)$ and $y_{t+1} \sim \mathrm{obs}(x_{t+1})$ for $t = 0, \ldots, T$.

TODO 2.4. The total loss is

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^{T} \big(L_t(q_t^b) + L_t(q_t^a) \big) + L_0(q_0^a),$$

minimized by L-BFGS over all parameters of **a**, **b**, **c** jointly.

2.4 Effect of the `deep` Parameter

Table 1 reports initial and final scores for different depths of the networks in modules **a** and **b**.

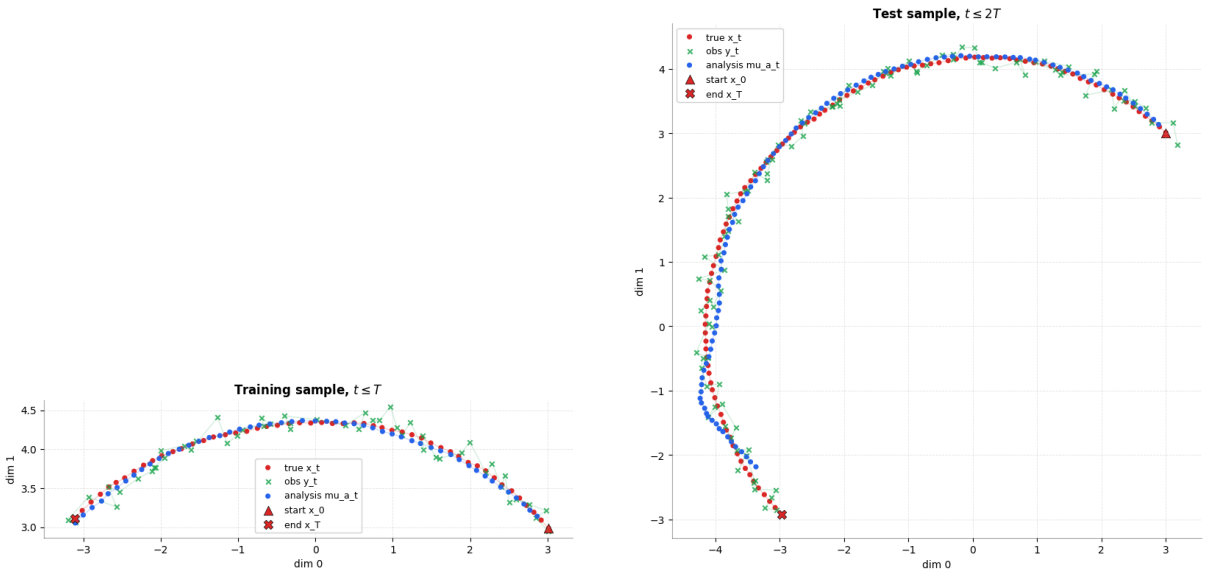
Table 1: Initial (i) and final (f) scores for different `deep` values on Linear 2D.

`deep`	RMSE_b		RMSE_a		LOGPDF_b		LOGPDF_a		LOSS	
	(i)	(f)	(i)	(f)	(i)	(f)	(i)	(f)	(i)	(f)
1	0.0136	0.0571	0.0493	0.0489	5.3815	3.0436	3.9036	3.3244	−9.2851	−6.3680
2	0.0608	0.0547	0.0244	0.0315	−3.4368	2.8683	3.9734	3.6020	−0.5367	−6.4703
5	0.0120	0.0322	0.0217	0.0286	5.5845	3.6024	4.8270	3.9144	−10.4115	−7.5167
10	0.0122	0.0292	0.0129	0.0280	5.6023	3.9822	5.2281	4.0531	−10.8304	−8.0353

Deeper networks consistently yield better final scores, with `deep`=10 achieving the best RMSE_a (0.028) and LOSS (−8.04). However, we see that the improvements show diminishing returns: the gap from `deep`=1 to 5 is much larger than from 5 to 10. For our simple 2D linear system, it seems like we do not need very deep networks to achieve good performance. The default `deep`=1 model is used for the rest of the experiments to save training time.

2.5 Trajectory Plots

The training-horizon plot (Figure 1a) shows that the analysis estimate μ_t^a follows the true state x_t closely, while observations y_t are noisier. The DAN has learned to denoise observations and maintain a coherent trajectory over the trained horizon.
On the extended test horizon (Figure 1b, $t \leq 2T$), we observe divergence of μ_t^a shortly after $t = T$. This indicates instability: the model is accurate on the trained horizon but accumulates error beyond it. The DAN has not learned a stable long-term dynamics model, and seems to rely on short-term corrections that fail to generalize.



(a) Training sample, $t \leq T$.

(b) Test sample, $t \leq 2T$.

Figure 1: State-space trajectories for Linear 2D. Red: true state x_t, green: observations y_t, blue: analysis mean μ_t^a.

3 Lorenz 96 (40D): Online Training

3.1 System and Setup

The Lorenz 96 model describes $n = 40$ coupled variables on a ring with the ODE

$$\frac{dx_i}{dt} = (x_{i+1} - x_{i-2})\, x_{i-1} - x_i + F, \qquad F = 8,$$

where indices are taken modulo n. This system is a well-known case of chaotic dynamics: small perturbations grow exponentially, which causes gradients to explode when backpropagated through thousands of steps via full BPTT. Instead, we train the DAN online with Adam, processing one timestep at a time and detaching the hidden state after each step to prevent gradient explosion.

Dynamics implementation For efficient vectorized computation, the ODE is expressed using a local stencil window $v_i = (x_{i-2}, x_{i-1}, x_i, x_{i+1})$ for each component i. The right-hand side decomposes into:

- a bilinear term $v_i^\top A\, v_i = x_{i-1}x_{i+1} - x_{i-2}x_{i-1}$ where A is a sparse matrix encoding the stencil interactions

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

- a linear term $b \cdot v_i = -x_i$
- a constant $c = F = 8$

The stencil vectors are assembled via `torch.roll` with `shifts=`$-w$ for each offset $w \in \{-2, -1, 0, 1\}$, reshaped to $(\text{mb}\cdot 40, 4)$, and the three terms are computed using `torch.nn.functional.bilinear` and `linear`. The `forward` method integrates this ODE using a 4th-order Runge-Kutta scheme (RK4).

Online training vs. full training Online training with Adam processes one timestep at a time. At each step t, a single (x_t, y_t) pair is generated, the loss $L_t(q_t^b) + L_t(q_t^a)$ is computed and minimized, and the hidden state is detached (`ha = ha.detach()`) to prevent gradients from flowing through the entire history. This truncated BPTT makes training feasible over $T = 10\,000$ steps through chaotic dynamics. In practice, we notice this method does take more time to converge than full training with L-BFGS. There is also added complexity, since we needed to implement a learning rate schedule and tune it for stable convergence.

3.2 Training and Test Scores

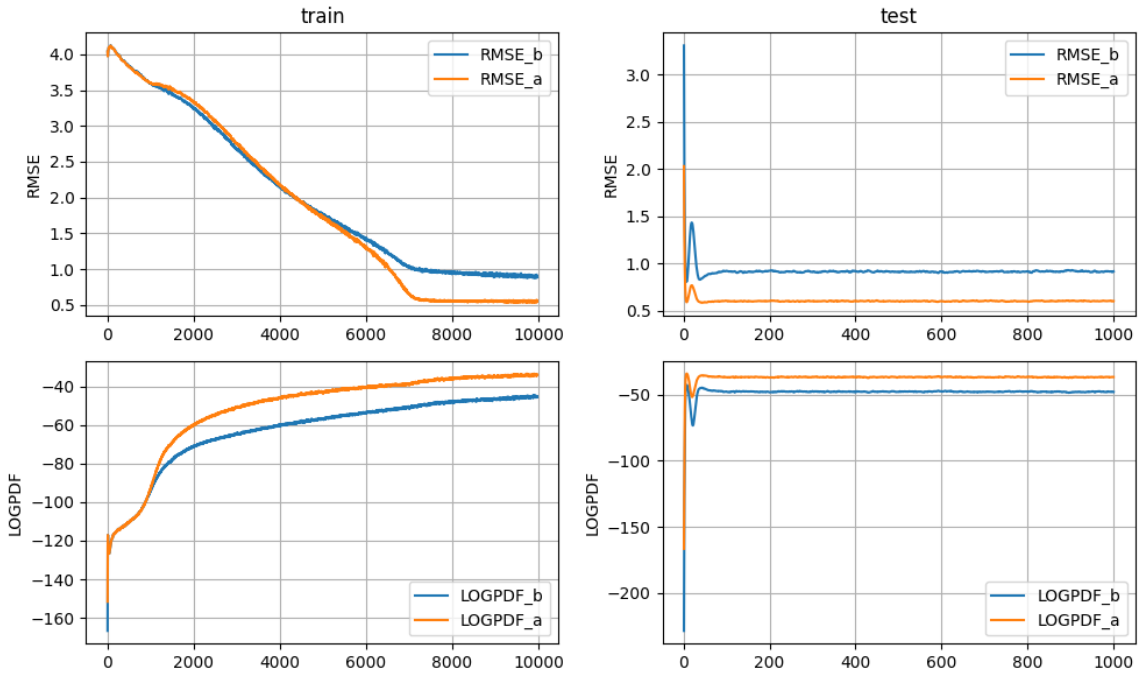


Figure 2: Training (left) and test (right) scores for Lorenz 96. Top row: RMSE, bottom row: LOGPDF. Prediction (b) and analysis (a) curves are overlaid.

The training curves (Figure 2, left) show steady improvement over ∼7000 steps, with a slowing rate of improvement afterwards. RMSE_a converges to approximately 0.65 and LOGPDF_a to -34. The test curves (Figure 2, right) show the DAN evaluated on new data with the hidden state initialized to zero. We notice a sharp transient regime in the first ∼50 steps. h^a starts at zero with no memory of the current state, so it would make sense that predictions are initially bad. With each new step, the analysis module **a** ingests a new observation and progressively adds information to the hidden state. Once h^a has accumulated enough context, the already trained pipeline of **a**, **b**, and **c** produces accurate estimates, and the metrics plateau: final values are $\text{RMSE}_a \approx 0.65$, $\text{RMSE}_b \approx 0.85$, $\text{LOGPDF}_a \approx -34$. The gap $\text{RMSE}_b > \text{RMSE}_a$ confirms that the analysis step always improves on the prior prediction by incorporating the observation, which is expected.

3.3 Sequence Plots

Figure 3 shows the analysis μ_t^a tracking the true chaotic state even at $t = 15\,000$, well beyond the training horizon $T = 10\,000$. Unlike the Linear 2D case, the online-trained DAN does not drift past its training horizon. It seems like the model now learns a per-step operator rather than memorizing a specific trajectory length. This is probably in great part thanks to the `ha.detach()` call that ensures the network cannot use the information from long-range gradients.

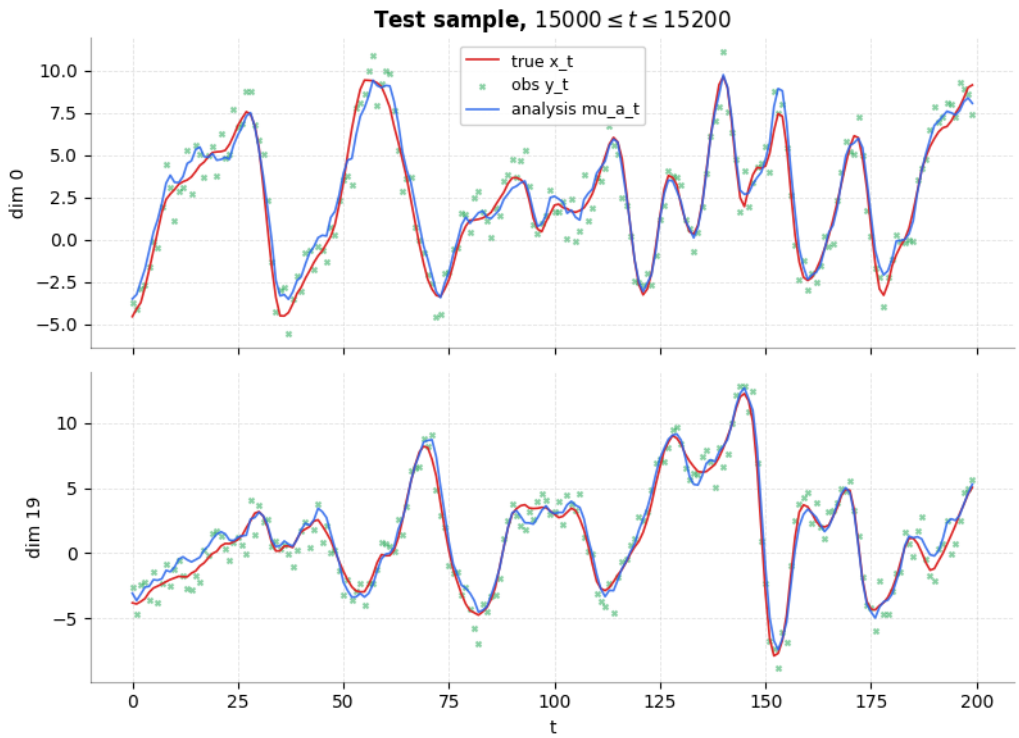


Figure 3: Test sample for Lorenz 96, dimensions 0 and 19, at $15\,000 \leq t \leq 15\,200$.

This performance did come at the cost of longer training: we computed 10 000 gradient steps, vs. ∼1200 closure evaluations for L-BFGS in the Linear 2D case. But for chaotic systems like Lorenz 96, as addressed in 3.1, it seems like online training with truncated BPTT is the only viable option.

4 Conclusion

We implemented and trained Data Assimilation Networks in two settings. Full-horizon training with L-BFGS is efficient for short, stable systems (Linear 2D) but produces models that fail to generalize beyond the training horizon. Online training with Adam and truncated BPTT performs well for chaotic systems (Lorenz 96) and yields analysis operators that generalize well beyond the training horizon, at the cost of slower convergence.